Step 4 — Graceful Degradation When One MCP Server Is Down — Solution

> Back to the step · Hint 1 · Hint 2

Story: this makes the team genuinely survive one specialist's MCP server being down. The failure is caught **inside** the Web Agent's own node, which writes a clear `web_unavailable` result instead of crashing; the Supervisor then reads that and routes straight to the Writer, skipping web entirely; and the Writer, told honestly, says the page couldn't be reached rather than staying silent. **If not:** one dead server would either crash the whole graph outright, or worse, quietly produce a report that pretends the web check happened when it never did.

The degradation is already in `run_specialist`

Look back at `src/mcp_client.py` from Step 1 — the whole connect-and-loop is wrapped:

```
    try:
        async with stdio_client(params) as (read, write):
            async with ClientSession(read, write) as session:
                await asyncio.wait_for(
                    session.initialize(), timeout=connect_timeout
                )
                # ... discover tools, run the loop ...
    except Exception as e:
        logger.warning(f"specialist for {args} unavailable: {e}")
        return "", True          # empty findings, unavailable=True
```

Two things make this resilient: `asyncio.wait_for(session.initialize(), ...)` so a server that never finishes starting times out instead of hanging forever, and the `try/except` that catches **inside the node's own async call** and returns `("", True)` — the exception never escapes to crash the graph. (This is Project 8 Step 4's exact instinct, reused.)

The pieces that carry the flag through

- `web_node` already writes `web_unavailable` into state (Step 2).
- `decide_next` already routes straight to `writer` once `web_unavailable` is true (Step 2, case 4) — no looping on a dead server.

- `run_writer` already reads `web_unavailable` and writes "(the web status page could not be reached)" into the report (Step 3).

So **no new wiring is needed in Step 4** — the flag was threaded through from the start. This step is about *proving* it: run the team with the web server's launch command broken and watch a complete, honest report come out.

`main.py` (final demo, with a broken web server)

```
# main.py  (final)
import asyncio
import sys

from src.config import load_config
from src.exceptions import MissingConfigError
from src.graph import build_graph
from src.models.llm import create_client
from src.state import start_state

TASK = ("Check my notes for Project Atlas, fetch its status page too, and "
        "give me one combined report.")


async def run_demo():
    try:
        config = load_config()
    except MissingConfigError as e:
        print(f"Setup problem: {e}")
        sys.exit(1)
    client = create_client(config)
    app = build_graph(client, connect_timeout=config.connect_timeout)
    final = await app.ainvoke(start_state(TASK))
    print("Routing:", final["routing_log"])
    print("\n" + final["report"])


def main():
    asyncio.run(run_demo())


if __name__ == "__main__":
    main()
```

(To see degradation live, point the web server's args at a script that doesn't exist in `src/agents/web_agent.py` — the team finishes from the notes alone and the report names the

gap.)

The integration test (the whole crew, live)

```
# tests/integration/test_live_crew.py
# Launches the real MCP servers and the model. Run from the repo root:
#   python -m tests.integration.test_live_crew
import asyncio

from src.config import load_config
from src.graph import build_graph
from src.models.llm import create_client
from src.state import start_state

TASK = ("Check my notes for Project Atlas, fetch its status page too, and "
        "give me one combined report.")


async def test_both_specialists_report(client):
    app = build_graph(client)
    final = await app.ainvoke(start_state(TASK))
    used = final["routing_log"]
    report = final["report"].lower()
    ok = "notes" in used and "web" in used and "writer" in used
    if "atlas" not in report:
        ok = False
    result = "WRONG"
    if ok:
        result = "OK"
    print(f"case 1 (both specialists -> one report) -> {result}")


async def test_web_down_degrades(client):
    # build a graph whose web agent points at a server that does not exist
    app = build_graph(client, web_args=["servers/does_not_exist.py"])
    final = await app.ainvoke(start_state(TASK))
    ok = final["web_unavailable"] is True
    report = final["report"].lower()
    if "notes" not in final["routing_log"]:
        ok = False
    if "reach" not in report and "unavailable" not in report:
        ok = False
    result = "WRONG"
    if ok:
        result = "OK"
```

```
    print(f"case 2 (web down -> honest degraded report) -> {result}")


async def main():
    client = create_client(load_config())
    await test_both_specialists_report(client)
    await test_web_down_degrades(client)


if __name__ == "__main__":
    asyncio.run(main())
```

For the web-down test to work, give `build_graph` an optional `web_args` and pass it to the web node's specialist call:

```
# src/graph.py  (final: make the web server path overridable for the test)
def build_graph(client, connect_timeout=10.0, web_args=None):
    if web_args is None:
        web_args = ["servers/web_server.py"]

    async def web_node(state):
        from src.mcp_client import run_specialist
        from src.agents.web_agent import WEB_SYSTEM
        findings, unavailable = await run_specialist(
            client, "python", web_args, WEB_SYSTEM, state["task"],
            connect_timeout,
        )
        return Command(goto="supervisor", update={
            "web_findings": findings, "web_unavailable": unavailable,
        })
    # ... the rest of build_graph unchanged ...
```

Expected output (real servers + model):

```
case 1 (both specialists -> one report)     -> OK
case 2 (web down -> honest degraded report) -> OK
```

That's the whole team: a Supervisor that only routes, two specialists each owning their own MCP connection, a Writer that synthesizes one report, and a dead server that degrades the team into an honest partial report instead of crashing it.